Project Documentation: Multi-Threaded Port Scanner

1. Overview

This Port Scanner is a high-performance network reconnaissance utility. It identifies open TCP ports on a target host, assisting in network administration and security auditing by mapping active services.

2. Technical Architecture

Core Components:

- **Socket Programming:** Utilizes the Python `socket` library to perform TCP connect scanning.
- **Concurrency Model:** Employs `threading` combined with a Queue to enable parallel scanning, drastically reducing execution time.
- **CLI Interface:** Built with `argparse` to provide a professional user experience for defining targets and port ranges.

3. Operation Workflow

The scanner operates via a **TCP Connect Scan**. It initiates a standard three-way handshake:

[Image of TCP three-way handshake process]

1. The client sends a SYN packet to the target port.
2. If the port is listening, the target responds with a SYN-ACK.
3. The client sends an ACK to complete the connection (and then immediately closes it).

If the connection attempt returns a success status (`result == 0`), the port is identified as **OPEN**.

4. Performance & Scalability

By default, scanning ports sequentially is inefficient due to network latency. The implementation uses a thread pool pattern:

- **Queueing:** Ports are pushed into a Queue.
- **Workers:** Multiple threads pull ports from the queue simultaneously.
- **Efficiency:** The `settimeout(0.5)` configuration balances speed and accuracy.

5. Ethical Usage & Legal Notice

This tool is designed strictly for educational and authorized professional security testing. Unauthorized scanning of networks, servers, or devices you do not own is illegal and unethical. Ensure you possess explicit written permission before executing this tool on any target environment.